

File Assembly and Collision Resolution — An Engineering Walkthrough

The same material as the academic monograph `AssemblyGuide.Academic.en.md`, minus formalism for its own sake: shorter notation, more "why it is built this way" and "how it behaves on a real stream". Section numbering matches in both versions, so the cross-references are interchangeable. For context (packet format, the H5/D3 hashes, accumulation) see `AlgorithmGuide.en.md`. The most detailed from-scratch walkthrough is `AssemblyGuide.Tutorial.en.md`.

1. What We Are Actually Assembling

After scanning, the decoder has:

- the file header: `N` data volumes, `M` ECC volumes, the file size, the SHA-256 of the whole file (`H3`), and `H5` — the seed of the sector hash;
- the received 64-byte payloads filed by sector number. One number may hold several **different** payloads — the versions; each carries a counter of how many times it arrived.

Assembly means: pick one payload per number, concatenate the first `N` volumes, trim to the file size, and compute the SHA-256. Match with `H3` — the file is ready. No match — take another combination. The key property: there is a single, final check, and it does not depend on how the packets arrived. So the outcome is always either a file bit-for-bit equal to the original, or an honest refusal (`null`). The state of "almost assembled, ship it anyway" does not exist, period.

2. Where Two Versions of One Sector Come From

- **Ordinary repeats are not a collision.** The same payload came again → the version's counter grew, nothing to search.
- **A random hash collision.** The sector check is 72 bits of SHA-256; it does not happen in practice, but it is theoretically possible.
- **A forgery.** H5 travels in the stream in the clear, so anyone who intercepted the stream can generate "valid" sectors with arbitrary content. The whole machinery exists for exactly this case.

A forgery cannot pass the final check — that would need a SHA-256 preimage. The worst it can do: an assembly refusal or search time burned. The defense has three layers: counters (repeats strengthen the truth), a search over equally-plausible candidates, and the final hash.

3. Where It All Lives

A file slot is a dictionary "sector number → version list". A version is a payload plus a counter. Two rules that always hold:

1. **The list is sorted by counter, descending.** The first element is the most confirmed; it is the default pick.
2. **Equal counters are never swapped with each other.** Inside an equally-probable group the order is stable; only the rotation knows how to change it (section 11), and only within the group.

An implementation detail worth knowing: a choice point holds a **live reference** to the slot's version list, not a copy. Rotation moves the slot's real lists — that is cheaper, and anyone wanting a snapshot calls `GetSectorVersions` and gets payload copies.

4. Receiving a Sector Copy (`AddSector`)

What happens for every received 75-byte sector packet:

1. Input checks: the number is within `0..N+M-1`, the payload is exactly 64 bytes. The hash has already proven authenticity — these checks merely guard the API against bad calls.
2. A new number — create the list, put the version in with counter 1.
3. Otherwise search linearly for a byte-equal payload:
 - found — counter `+1` (capped at `int.MaxValue`, so no overflow) and **bubbling up**: the version rises while a strictly less confirmed one sits above it. It never climbs past equals — rule 2 of section 3.
 - not found — a new version goes to the **end** of the list (every existing version has a counter ≥ 1 , so the sort holds).

Why a linear search instead of a payload dictionary: in 99.9% of cases the list holds one version, and a single 64-byte compare is cheaper than maintaining a hash table per sector. Memory is saved too. A side effect of bubbling — the list "leader" is always the most confirmed version. The whole "preferred selection" logic below rests on it.

5. The Assembly Plan

TryAssemble

1. selected $\leftarrow$ versions[0] of every number (the most confirmed)
2. points $\leftarrow$ numbers where several versions share the top counter
3. no such numbers?
 - yes $\rightarrow$ a single attempt: direct assembly $\rightarrow$ RS // sections 6-7
 - no $\downarrow$
4. C $\leftarrow$ the product of the equally-probable counts (clamped at long.Max)
5. attempt #0 (the current selection) timed, t_0
6. exhaustive estimate = $t_0 \cdot C$
7. $C \leq 100,000$ and the estimate ≤ 30 s?
 - yes $\rightarrow$ exhaustive search (section 10)
 - no $\rightarrow$ rotation (section 11)
8. still $\downarrow$ and ECC exists $\rightarrow$ volume subset search (section 12)

Both strategies run the **same** attempt on every step (section 6, then 7) and differ only in how they prepare the next version set. Cancellation and the time budget are checked between attempts. Stage 3 finishes off what they cannot handle — no collisions, with a budget of its own (section 12).

6. A Single Attempt: Direct Assembly

Boring and fast: if all N data volumes are present in the selection — concatenate $N \cdot 64$ bytes, trim to the file size, hash, compare. One attempt costs a concatenation plus one SHA-256. The direct path covers the case of no losses, or losses covered by repeats.

7. A Single Attempt: the RS Patch

Direct failed and ECC is configured:

1. Build the "which volumes are in" map over the whole $N+M$ line.
2. Fewer than N volumes in place — nothing to recover from, fail.
3. Call the Reed–Solomon decoder: it returns N data volumes — the intact ones as is, the lost ones recovered from ECC. The recovery condition is simple: lost data $\leq$ available ECC.
4. Concatenate the result and check with the final hash.

Two things that are easy to miss:

- **ECC volumes are sectors too and can collide as well.** A forged ECC corrupts the recovery, the final hash mismatches — so the search must click through ECC-number versions too. It does: the selection is built over the whole $N+M$ line.
- RS runs in **every** search attempt. The search only varies the contested sectors; the holes are recovered by ECC each time.

8. Which Versions Take Part in the Search at All

A branch point is a number where two or more payloads share the top counter. The search walks **only** these equally-probable heads. Versions with smaller counters are tried by neither exhaustive search nor rotation — as far as this assembly call is concerned, they do not exist.

If the truth ends up in the minority (the forgery got duplicated more), the current assembly refuses. That is a deliberate trade: the chance of a hash-valid forgery is tiny, while a code path for "but what if the minority is right" would cost an exponent. The remedy is systemic: keep receiving — repeats will pull the truth up to a tie, and the next assembly call will see a choice point. Or the sector is not needed at all — RS will cover it.

9. How Many Combinations and Which Search to Pick

The combination count is the product of the head sizes. It is computed in `long`; if the product threatens to overflow, `long.MaxValue` comes back (shown as "> long.Max" in reports). The clamp exists so the limit comparison never blows up on arithmetic.

Exhaustive search is allowed when both hold:

1. no more than 100,000 combinations (a hard limit);
2. the time estimate fits into 30 seconds.

The estimate is not from thin air: the zero attempt is timed with a stopwatch, its duration multiplied by the combination count. An attempt includes SHA-256 and possibly RS — the price depends on the file size, which is why we measure the fact rather than trust a formula. Either condition fails — we rotate.

10. Exhaustive Search: the Odometer

Combinations are encoded as an index array with moduli — a "drum counter", like the odometer on old meters: the right drum turns first; at its limit it resets to zero and pushes the left one. All drums exhausted — the combinations are over.

```
attempt #0 has already been run by the caller
  for i = 1 .. C-1:
    check cancellation and the budget (30 s)
    click the odometer one notch
    substitute the choice-point heads into the selection
    run the attempt (sections 6-7); success — return the file
  fail
```

The substitution touches only the local selection; the slot's version order is untouched. The guarantee is simple: if the correct combination lies within the equally-probable heads and the budget holds — the exhaustive search finds it.

11. Rotation: the Heuristic for Heavy Cases

When there are too many combinations, walking everything is not an option. An observation from attack practice: a forgery is duplicated into all attacked sectors **identically**, i.e. the rank displacement of the truth is the same everywhere. Rotation checks exactly such "synchronous" shifts.

Mechanics: after every failed attempt, in every contested sector the first equally-probable variant moves to the end of the equally-probable head. One step — one shift everywhere.

The analogy is several cyclic lists of different lengths turning in sync. The picture repeats after the LCM of the lengths; at step `k` the chosen combination is "`k` modulo the length" in every list — a diagonal. From this everything follows:

- **Pairwise-coprime lengths** → the LCM equals the product → rotation visits every combination, just in a different order than the odometer. Equivalent to exhaustive search.
- **Common divisors present** → some combinations are unreachable. Example: three heads of 2 variants each — $C = 8$, but the LCM is 2; rotation checks only the two "pure" combinations and not a single mixed one. That is the price of speed.
- A **systematic forgery** (the same shift in all attacked sectors) always lies on a diagonal — caught in one of the first attempts.

The state count is clamped at 100,000; the budget and cancellation are checked at every step. Rotation moves the slot's real lists, but that is safe: a permutation inside an equally-probable head means nothing to future assemblies — any starting permutation is equally valid.

12. Silent Corruption and the Volume Subset Search

Both searches from sections 10–11 assume the damage is **visible**: a collision exists, so there is something to iterate. But what if a volume is formally honest — the number is intact, the 72-bit sector hash matches — while the payload is corrupted (a random corruption slipped past the check, or a forgery was crafted together with its hash)? One version, no contest, RS with the full map trusts the corrupted volume, and the file hash never matches — with zero leads.

Stage 3 flips the problem: if there is nothing to vary among versions, vary the map. Suspicious volumes are marked erased — as if they never arrived — and RS recovers them from the rest. Exclude exactly the bad one, and the truth gets recovered and confirmed by the final hash.

The order of attempts (the `SubsetMaskPlanner` — pure functions, deterministic, easy to test):

1. **Base** — all collision slots at once: instead of clicking through their versions one by one, erase the sectors and recover them from ECC. If that does not fit the ECC budget — the plan is empty and the stage honestly gives up.
2. **Level 1** — exclude each present uncontested volume one at a time, starting with the most suspicious: fewer confirmations — earlier; ties — in a pseudo-random order seeded from H5 (the packet arrival order does not affect the outcome, and a repeated call reproduces the same plan).
3. **Levels 2–3** — pairs and triples from the 64 most suspicious volumes (a shortlist, configurable).

Why a separate cap on E — the number of erased data volumes (32 by default): the price of an attempt is the Gaussian inversion $\sim E^2 \cdot N$. With $E \leq 32$ an attempt costs milliseconds even on the largest field ($T = 65,535$); with $E \geq 128$ it already costs seconds. That is why the erased-volume count is limited separately from the attempt count (100,000) and the budget (30 s); whichever fires first wins.

When it helps: damaged media and "hash-consistent" forgeries without collisions — one, two, three corrupted volumes within the ECC budget. When it does not: massive damage that does not fit the ECC — an honest refusal.

13. The Final Check and Cleanup (`TrimAndVerify`)

The `N·64` buffer is trimmed to the file size (the last volume was encoded with zero padding), hashed, compared with `H3`. A match — the file goes out. No match — the buffers are **wiped with zeros** and a refusal is returned. The wipe is not paranoia: a failed attempt may contain a forgery, and there is no reason to let it linger in the heap.

The bottom rule: assembly never hands out an unchecked result. A refusal is a refusal, not "roughly it".

14. What Is Visible from Outside

Member	What it shows
<code>HeaderReceptionCount</code>	how many header copies were caught
<code>ReceivedSectorCount</code>	how many numbers are covered by at least one version
<code>ReceivedSectorCopyCount</code>	how many sector copies in total (sum of counters)
<code>CollisionSectorCount</code>	how many numbers have competing versions
<code>Coverage</code>	the percentage of covered numbers of <code>N+M</code>
<code>FormatValidityMap</code>	a string map:  received,  collision,  hole
<code>BuildCollisionMap</code>	number → how many versions
<code>GetSectorVersions(n)</code>	a snapshot of the sector's versions in preference order

Progress is throttled to whole percents and never shows 100 until the file is actually assembled. Inside the search, progress walks combinations ("exhaustive search: k / C"), not bytes.

15. Limits and Settings

Setting	Default	What it caps
<code>MaxExhaustiveCombinations</code>	100,000	the ceiling of combinations for exhaustive search
<code>MaxHeuristicAttempts</code>	100,000	the ceiling of rotation states (including zero)
<code>TimeBudget</code>	30 s	a soft budget; checked between attempts

Stage 3 (section 12) lives in the `SubsetSearch` group:

<code>SubsetSearch</code> setting	Default	What it caps
<code>MaxAttempts</code>	100,000	the stage's attempt ceiling
<code>TimeBudget</code>	30 s	the stage's soft budget
<code>MaxErasedDataVolumes</code>	32	erased data volumes per attempt (the cap on E)
<code>ShortlistSize</code>	64	the shortlist for exclusion pairs and triples
<code>MaxExtraExclusionLevel</code>	3	the maximum size of extra exclusions

Overflows never throw during counting: combinations are clamped at `long.MaxValue`, the LCM at the attempt limit, and the per-level binomial coefficients at `long.MaxValue` too.

16. Post-Mortems

16.1. Clean reception

A 100-byte file without ECC: two sectors, each received once. No contest → a single attempt, direct assembly, the hash matches. Done.

16.2. One contested sector, two variants

Sector 0: variants `A` and `B`, three confirmations each. $C = 2$ → exhaustive: attempt #0 takes `A`, attempt #1 takes `B`. The truth shows up by the second attempt at the latest.

16.3. Two contested sectors: 3×2

$C = 6$, $\text{LCM} = 6$ — coprime. Exhaustive and rotation walk the **same** set of six attempts, just in a different order (the odometer by rows, rotation by diagonals). Both are complete.

16.4. Three contested sectors: $2 \times 2 \times 2$

$C = 8$, but suppose the time did not fit → rotation: $\text{LCM} = 2$, only `A+X+P` and `B+Y+Q` are checked. The six mixed combinations are skipped; if the truth is there — an honest refusal. The price of speed: a full pass would cost four times more.

16.5. A forgery won the confirmations

Sector 5: truth `A` (twice), forgery `B` (five times). The list head is a lone `B`, no contest, the selection always takes `B`, the hash mismatches, refusal. No search can help — `A` is outside the search field (section 8). What to do: keep receiving; repeats will pull `A` up to a tie, and the next assembly call will start branching.

16.6. A silently corrupted volume

`N = 64`, `M = 8`. Volume 7 is corrupted yet passed the sector check; one version. Stages 1–2: no contest, the single attempt hands RS the full map — RS trusts volume 7, the hash mismatches. Stage 3: no collisions, the base is empty; level 1 excludes volumes one by one — on the mask `{7}` RS recovers the truth and the hash matches. On the order of N cheap attempts, each a single inversion at $E = 1$.

17. What Is Guaranteed and What Is Not

Guaranteed:

1. A file was returned — its SHA-256 equals `H3`, i.e. it is the original bit-for-bit (within the strength of SHA-256).
2. All data volumes received unambiguously — success on the first attempt.
3. Combinations $\leq 100,000$ and the time fits — the correct set within the equally-probable heads will be found.
4. Coprime heads — rotation visits everything, equivalent to exhaustive search.
5. There are no false results: either a proven file, or `null`.

Not guaranteed (heuristics):

6. With common divisors, rotation sees only the LCM diagonals.
7. Minority versions are not checked at all in this call.
8. Fitting into 30 seconds is an empirical estimate from the zero attempt's timing.
9. A single silent corruption outside collisions is caught by stage-3 level 1 — as long as the ECC budget and the limits hold.
10. Pairs and triples of silent corruptions — only within the shortlist and the exclusion level.

Numbers to remember: the sector check is 72 bits, the header check 192 bits, the arbiter is SHA-256; exhaustive search up to 100,000 combinations, rotation up to 100,000 states, subset search up to 100,000 masks with the cap on $E = 32$, budgets of 30 s each. All configurable.